

RELACIÓN DE EJERCICIOS 05

Relación 10 Programación orientada a objetos

Clases, constructores, atributos, métodos, herencia y composición como preparación para el Python de Odoo.

5 ejercicios

class · self · herencia

Sin Odoo

Relación 10 · Programación orientada a objetos

Clases, constructores, atributos, métodos, herencia y composición como preparación para el Python de Odoo.

Instrucciones

Crea un archivo .py diferente para cada ejercicio. Resuelve los ejercicios individualmente usando los archivos base de 04_Archivos_para_practicar. Los archivos base contienen el enunciado abreviado y el punto de partida, no la solución. No necesitas acceder a Odoo.

Cómo ejecutar y entregar

```
py -3 R10E1.py
```

Cambia el nombre de archivo para cada ejercicio. Entrega los archivos .py por separado. Usa datos ficticios. Cuando el ejercicio requiera repetición, construye el bucle con una condición explícita; evita while True combinado con break.

Ejercicio 01 · Clase Producto

Crea una clase Producto con atributos nombre, precio y stock. El constructor recibirá los tres valores. Añade un método mostrar() que presente una ficha del producto.

Comprobación

Crea al menos dos objetos Producto y llama a mostrar() en ambos.

Ejercicio 02 · Métodos de inventario

Amplía una nueva clase Producto para incluir vender(cantidad) y reponer(cantidad). vender() solo debe reducir el stock si hay unidades suficientes y devolver True; si no, devuelve False. reponer() incrementa el stock.

Comprobación

Prueba una venta posible y otra superior al stock disponible.

Ejercicio 03 · Clase Cliente

Crea Cliente con nombre, email y ciudad. Añade `cambiar_ciudad(nueva_ciudad)` y `ficha()`, donde `ficha()` devuelva una cadena con los datos del cliente. No imprimas directamente dentro de `ficha()`.

Comprobación

Cambia la ciudad de un objeto y muestra la ficha antes y después.

Ejercicio 04 · Herencia Empleado - Comercial

Crea Empleado con nombre y salario. Crea Comercial(Empleado) con un atributo adicional `ventas`. Añade `calcular_bonus()` que devuelva el 5 % de ventas y un método `resumen()` que incluya salario y bonus.

Comprobación

Usa `super().__init__()` para inicializar la parte heredada.

Ejercicio 05 · Pedido y líneas de pedido

Crea una clase `LineaPedido` con `producto`, `precio` y `cantidad` y un método `subtotal()`. Crea una clase `Pedido` que guarde cliente y una lista de líneas. Añade `agregar_linea(linea)` y `total()` para sumar los subtotales.

Comprobación

Crea un pedido con al menos tres líneas y comprueba que `total()` coincide con la suma manual.

Revisión antes de entregar

- El archivo se ejecuta sin errores de sintaxis.
- La sangría es coherente: cuatro espacios por nivel.
- Los nombres de variables y funciones usan `snake_case`.
- No has pegado todos los ejercicios en un único archivo.